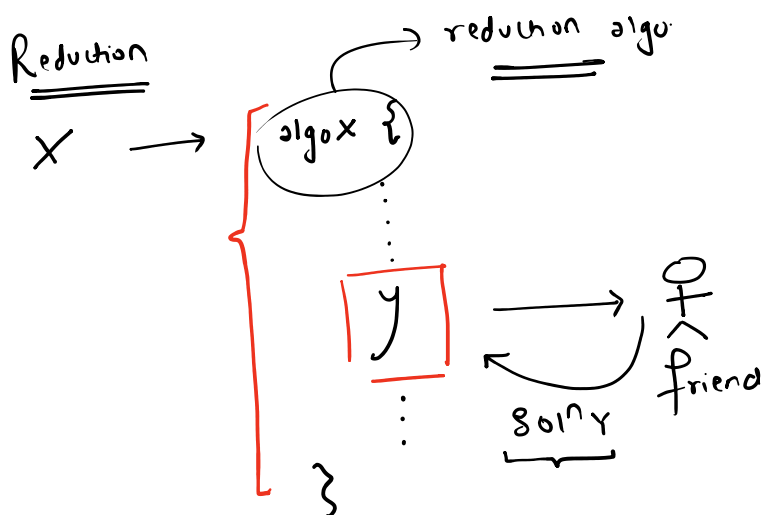


➤ Reduction

⇒



$f_x \{$
 $= \dots$
 g_y
 $\dots$
 $\}$

➤ Recursion

It is a kind of **Reduction**.

Suppose you are given a problem X for which you want to design a recursive algorithm then recursion says

⇒

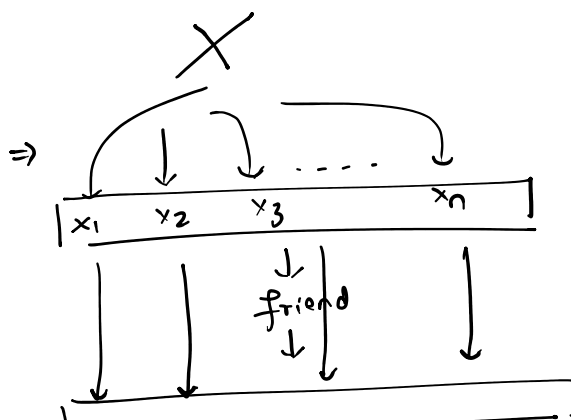
{

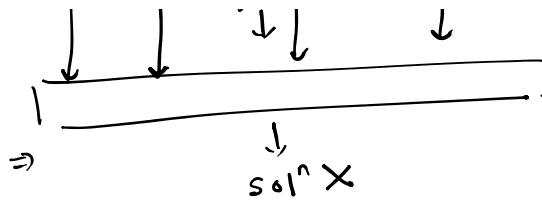
- If you can solve X directly then solve it directly. **[Base Case]**
- Otherwise, reduce X into one or more **smaller** instances of the same problem (sub-problem). **[Recursive Case]**

⇒

⇒

f_x





$f_x \{$

...

$f_{x1} \dots f_{xn}$

...

...

$\}$

Factorial of a Number

Given a non-negative number n , design a recursive **algorithm** to compute $n!$.

$$n = 5$$

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$$

$$= 5 \cdot (4 \cdot 3 \cdot 2 \cdot 1)$$

$$n! = n \cdot \underbrace{(n-1)!}_A \quad \text{friend}$$

$$= n \cdot A$$

Recurrence Relⁿ

$$\Rightarrow f(n) = n \cdot \underbrace{f(n-1)}_A$$

Recursive Case

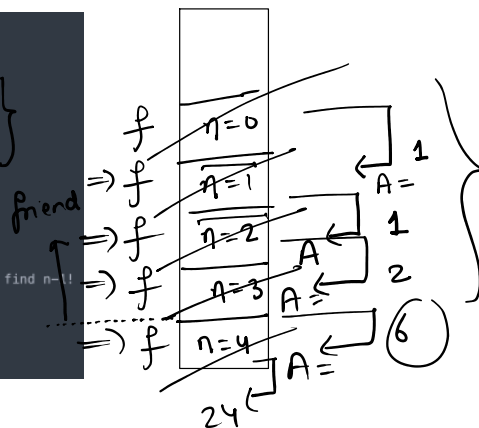
it is a fn that finds $(n-1)!$

$$f(5) = 5 \cdot \underbrace{f(4)}_{4 \cdot \underbrace{f(3)}_{3 \cdot \underbrace{f(2)}_{2 \cdot \underbrace{f(1)}_{1 \cdot f(0)}}}}$$

$$f(0) = 0 * \underbrace{f(-1)}_{\text{invalid fn call}} \quad \underline{\text{fail}}$$

$$\boxed{f(0) = 1} \quad \text{Base Case}$$

```
int f(int n) {
    // base case
    if (n == 0) {
        // f(0) = find 0!
        return 1;
    }
    // recursive case
    // f(n) = find n!
    // 1. ask your friend to find n-1!
    int A = f(n-1);
    return n * A;
}
```



time:

#rec. calls * time spent in each call

Base Case: $\underbrace{f_0, f_1, \dots, f_{n-2}, f_{n-1}, f_n}_{n \text{ calls}}$

$\therefore \text{const}$

$\therefore n \cdot \text{const}$

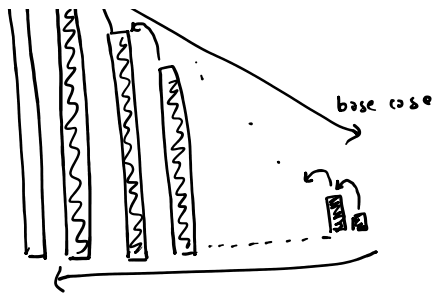
$\therefore O(n)$

max. no. of stack frames

space needed for each stack frame

$(n+1) \cdot \text{const} \therefore O(n)$

due to fn call str

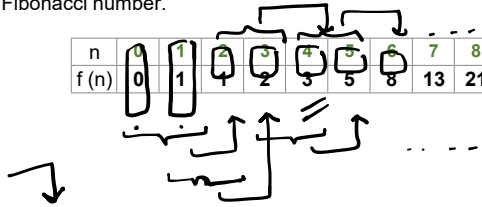


for each stack frame

Fibonacci Numbers

Given a non-negative number n , design a recursive algorithm to compute the n^{th} Fibonacci number.

n	0	1	2	3	4	5	6	7	8	9	...
f(n)	0	1	1	2	3	5	8	13	21	34	...



$$n^{\text{th}} \text{ fib. no.} = \underbrace{A}_{(n-1)^{\text{th}} \text{ fib. no.}} + \underbrace{B}_{(n-2)^{\text{th}} \text{ fib. no.}}$$

friend

it is a fn that finds the n^{th} fib. no.

$$\Rightarrow f(n) = \underbrace{f(n-1)}_{A / \text{friend}} + \underbrace{f(n-2)}_{B / \text{friend}}$$

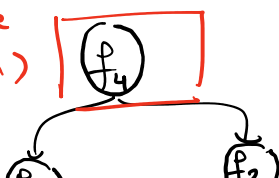
Base Case

$$f_0 = \underbrace{f_{-1}}_{\text{invalid}} + \underbrace{f_{-2}}_{\text{calls}}$$

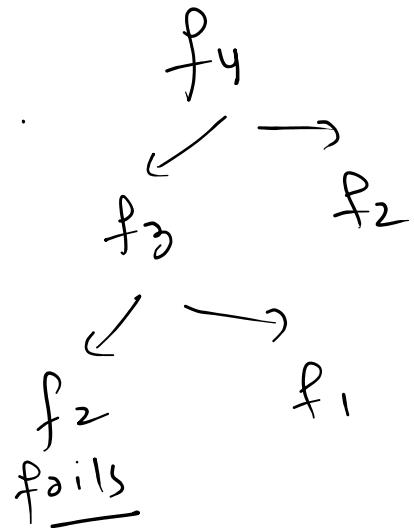
$$f_1 = f_0 + \underbrace{f_{-1}}_{\text{invalid calls}}$$

$$\begin{cases} f(0) = 0 \\ f(1) = 1 \end{cases}$$

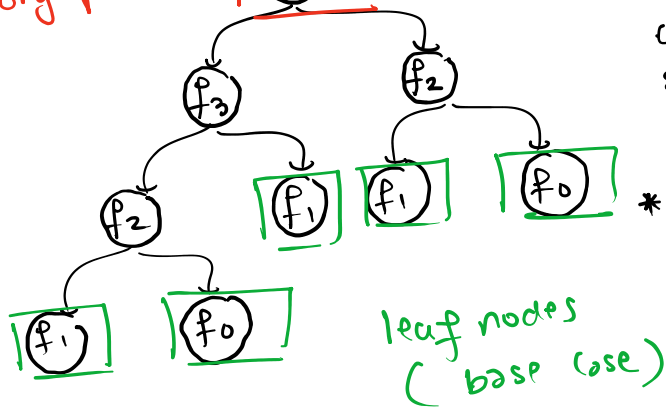
root node
(orig. problem)



* each node of recursion tree
corr. to a subproblem



Long run $\rightarrow$ 1992

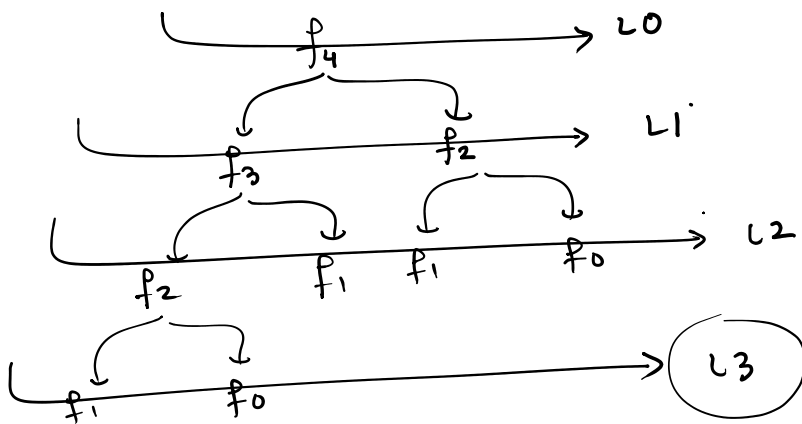


corr. to a subproblem

* total no. of nodes in revision tree

total no. of subproblems you are solving

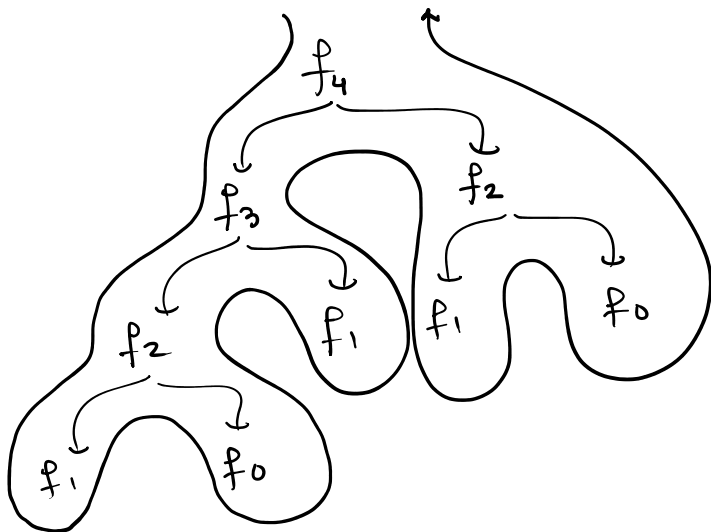
total no. of
rewrte calls



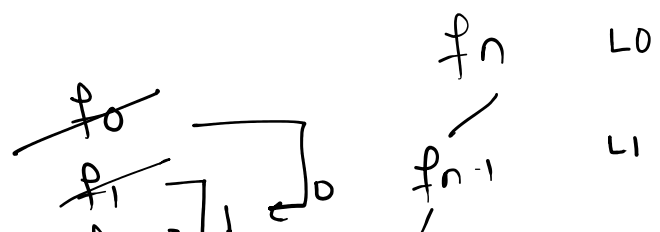
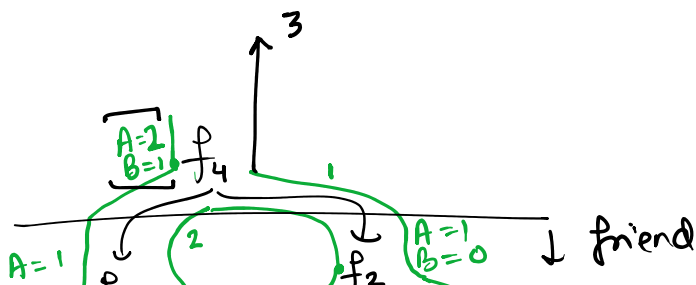
max-level = 3 $\therefore$
tree hgt = 3

Max. level
of tree
=
Max. depth
of a node

$\overline{\overline{\text{height of the tree}}}$



dfs



Multiply Numbers

⇒ Given two non-negative integers x and y , design a recursive algorithm to compute their **product**.

$$x = 3 \quad y = 4$$

$$x * y = 3 * 4 = 12$$

$$x * y = \underbrace{x + x + \dots + x}_{y \text{-times}}$$

$$= \boxed{x + A}$$

y-1 times
friend

$f(x, y)$ it is a fn that
cal. $x * y$ i.e.
it is a fn that
adds x , y times.

$$= x + \underbrace{f(x, y-1)}_{A / \text{friend}}$$

$$f(x, 0) = x + \underbrace{f(x, -1)}_{\text{invalid}} \quad \underline{\underline{\text{fails}}}$$

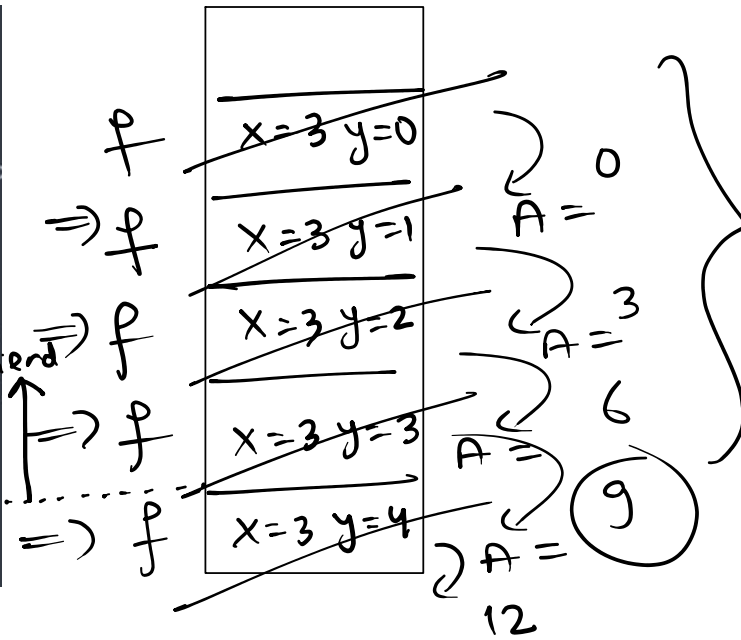
$$\boxed{f(x, 0) = 0} \quad \text{Base Case}$$

```

int f(int x, int y) {
    // base case
    if (y == 0) {
        // f(x, 0) = add
        return 0;
    }

    // recursive case
    // f(x, y) = add x, y
    // 1. ask your friend
    int A = f(x, y - 1);
    return x + A;
}

```

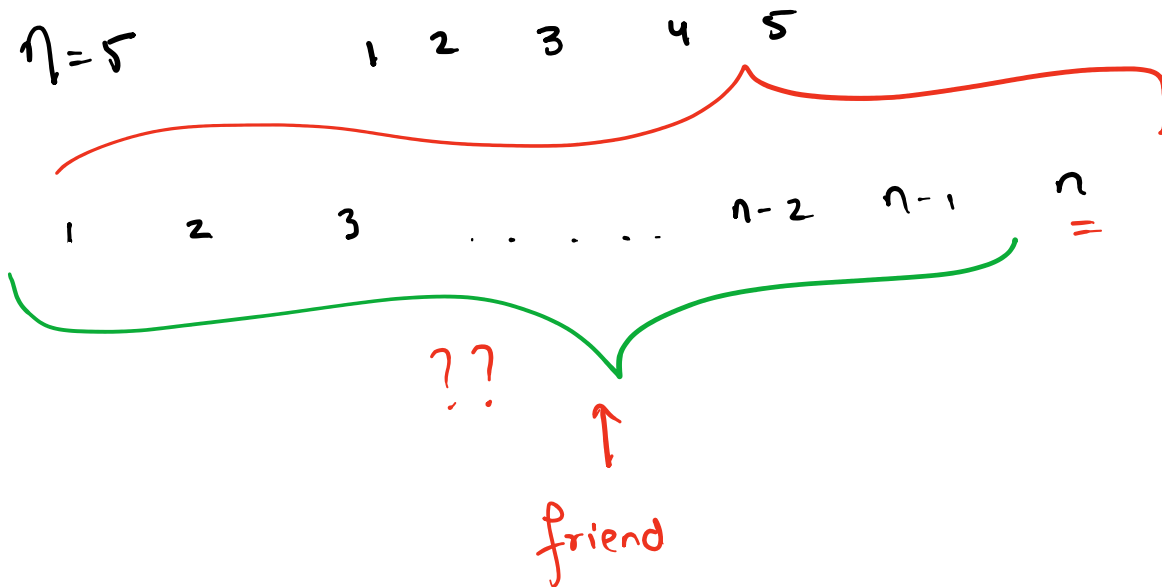


time: $O(y)$
 space: $O(y)$
 due to fn
 call stack

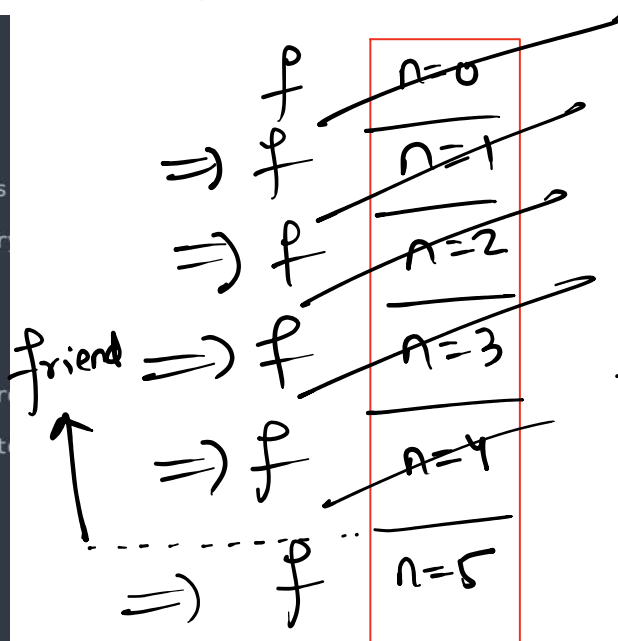
Print Increasing

Print Increasing

Given a **positive** integer **N**, design a recursive algorithm to print numbers from **1** to **N** in the **increasing** order.



```
void f(int n) {  
    // base case  
    if (n == 0) {  
        // f(0) = print nos  
        return; // mandatory  
    }  
  
    // recursive case  
    // f(n) = print nos. fr  
    // 1. ask your friend to  
    f(n - 1);  
    cout << n << " ";  
    return; // optional  
}
```



o/p: 1, 2, 3, 4, 5

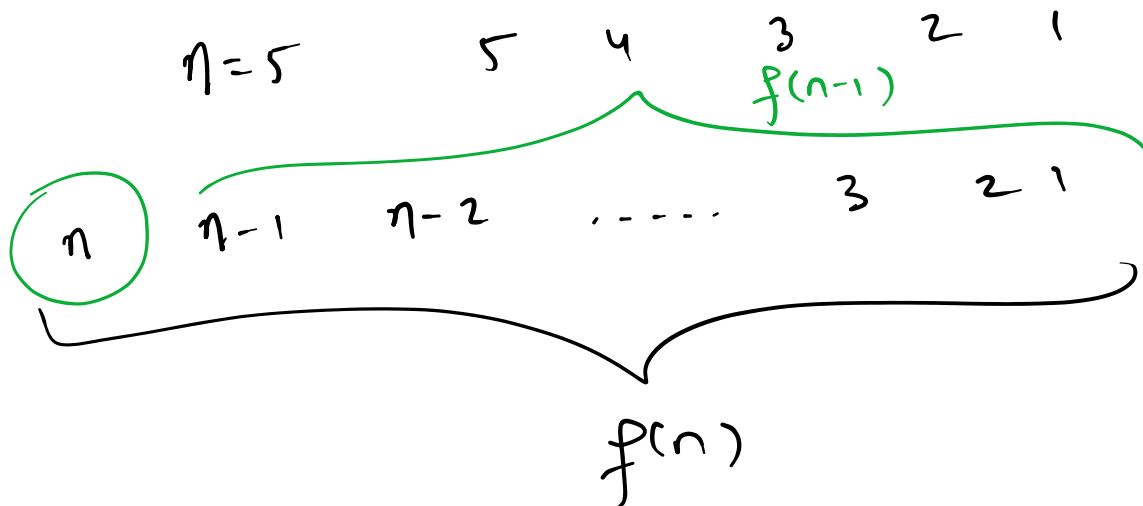
time: $O(n)$

space: $O(n)$ due to f_n call str

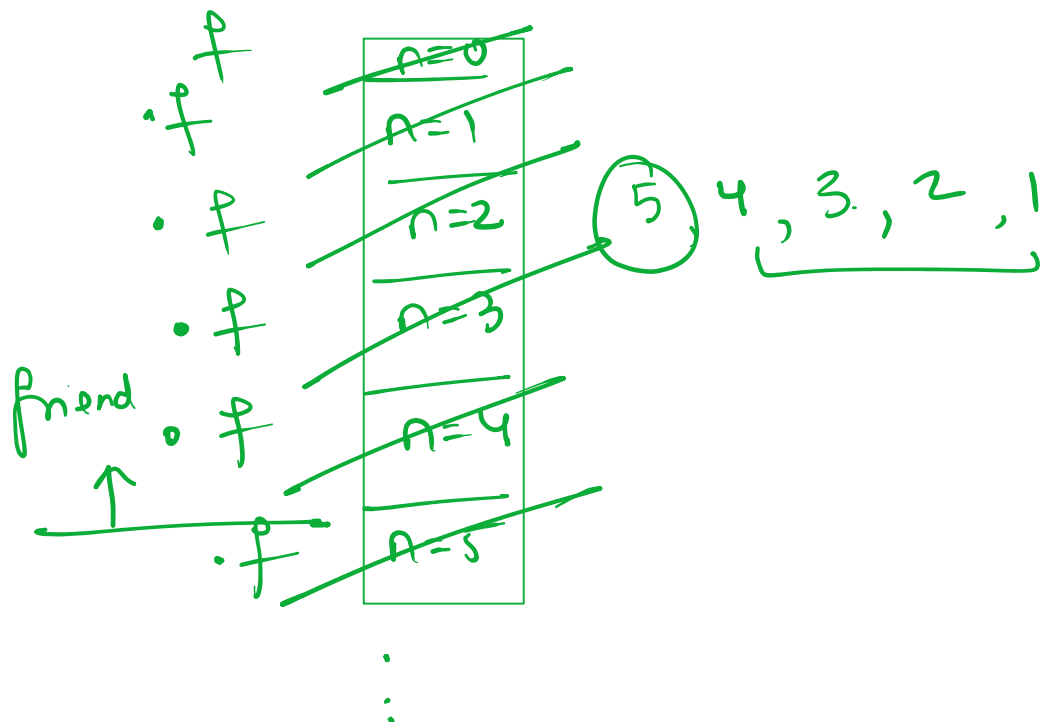
Print Decreasing

Print Decreasing

Given a **positive** integer **N**, design a recursive algorithm to print numbers from **N to 1** in the **decreasing** order.



```
void f(int n) {  
    // base case  
    if (n == 0) {  
        // f(0) = print nos  
        return; // mandatory  
    }  
    // recursive case  
    // f(n) = print nos. fr  
    cout << n << " ";  
    // 1. ask your friend to  
    f(n - 1);  
    return; // optional  
}
```



[HW] $3n + 1$ Sequence

<https://codeforces.com/group/MWSDmqGsZm/contest/223339/problem/Q>

Power

Given two **non-negative** integers x and y , design a recursive algorithm to compute x^y .

Example

Input: $x=2, y=3$
Output: 8

$$2^3 = 8$$

$y-1$ times $\Rightarrow A$ (friend)

$$x^y = \underbrace{x \cdot x \cdot x \cdots x}_{y \text{ times}}$$

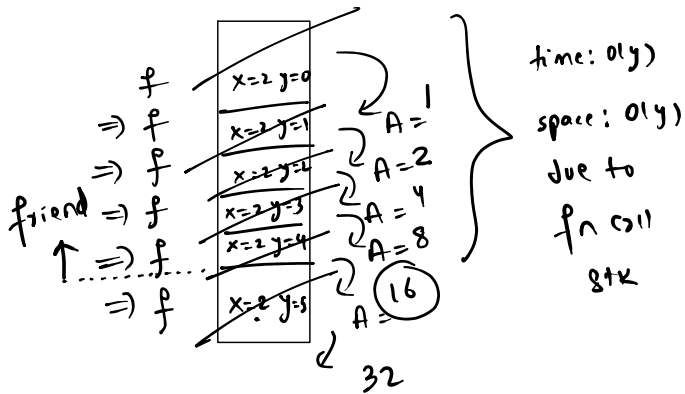
$$= x \cdot A$$

$f(x, y)$

$\hookrightarrow$ it is a fn that mul. x ,
 y times i.e it is a fn
that cal. x^y

$$= x \cdot \underbrace{f(x, y-1)}_{A/\text{friend}}$$

$$\boxed{f(x, 0) = 1}$$



$$\boxed{x^y = x \cdot x^{y-1}}$$

$A \cdot A$

$$x^y = \underbrace{x \cdot x^{y/2} \cdot x^{y/2}}_A$$

assuming y is even

$$3^6 = 3^3 \cdot 3^3$$

$$4^{10} = 4^5 \cdot 4^5$$

$A \cdot A \cdot x$

$$x^y = x^{y/2} \cdot x^{y/2} \cdot x$$

$$x^y = x^{y/2} \cdot x^{y/2} \cdot x$$

when y is odd

$$4^{10} = 4^5 \cdot 4^5$$

$$7^8 = 7^4 \cdot 7^4$$

$$\vdots$$

$$3^5 = 3^{5/2} \cdot 3^{5/2}$$

$$= 3^2 \cdot 3^2 \cdot 3$$

$$7^9 = 7^{9/2} \cdot 7^{9/2} \cdot 7$$

$$= 7^4 \cdot 7^4 \cdot 7$$

$$x^y$$

$$\downarrow$$

$$x^{y/2}$$

$$\downarrow$$

$$x^{y/4}$$

$$\downarrow$$

$$x^{y/8}$$

$$y = y/2^0 \Rightarrow 1$$

$$\downarrow$$

$$y/2 = y/2^1 \Rightarrow 2$$

$$\downarrow$$

$$y/4 = y/2^2 \Rightarrow 3$$

$$\downarrow$$

$$y/8 = y/2^3 \Rightarrow 4$$

$$\vdots$$

$$\boxed{y/2^k = 1} \Rightarrow k+1$$

0 ← Base Case

$$2^k = y$$

$$k = \log_2 y$$

$$\therefore \sim \log_2 y \cdot \text{const}$$

$$\therefore O(\log_2 y)$$

space;

$O(\log_2 y)$ due to
fn call stack

[HW] Count Good Numbers

<https://leetcode.com/problems/count-good-numbers/>

[R][HW] Generate Binary

Generate Binary

Given a position integer **N**, design a recursive algorithm to **generate its binary**

Example

Input : N = 20

Output : 10100

[R][HW] String to Integer

"123" → 123
(string) (int)

String to Integer

Given a string **str**, design a recursive algorithm to **convert** it to an integer.

Example

Input : str = "123"
Output : 123

$\left\{ \begin{array}{l} \text{stoi} : ("123") \rightarrow 123 \\ \hspace{15em} (\text{int}) \\ \text{stoll} ("123\dots") \rightarrow 123\dots \\ \hspace{15em} (\text{long long}) \\ \text{to_string} (123) \rightarrow "123" \\ \hspace{15em} (\text{int}) \hspace{5em} (\text{string}) \end{array} \right.$

[HW] Spell Number

Spell Number

Given a positive integer **N**, design a recursive algorithm to **spell** it.

Example

Input : N = 123

Output : one two three

i/p $\uparrow$ = 1 2 3

o/p one two three

[HW] Move X

Move X

Given a string "**str**", design a recursive algorithm to **move** all the '**x**' characters to the **end** of the given string such that **relative position** of **non-x** characters doesn't change.

Example

Input : str = "xaxbxc"

Output : "abcxxx"

Input : str = "axbxcx"

Output : "abcxxx"

"x a x b" $\Rightarrow$ "a b x x"

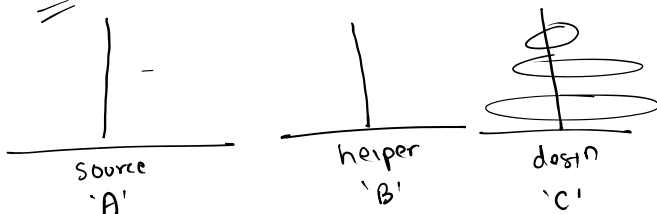
"c x a x b" $\Rightarrow$ "c a b x x"

Tower of Hanoi

Given a **three towers** and **N disks** of varying sizes that can **slide** onto any of the towers. Initially, all the **N disks** are placed in the decreasing order of their sizes on the **1st tower (source)** i.e. the **smallest** disk at its **top** and the **largest** disk at its **bottom**. Design a recursive **algorithm** for generating the steps **required** to move the **N disks** from the **source** tower to the **3rd tower (destination)** using the **2nd tower (helper)** such that at any point of time

- you can move a **one** disk from top of one tower to the top of another.
- you cannot place a **larger** disk on top of **smaller** disk

Example

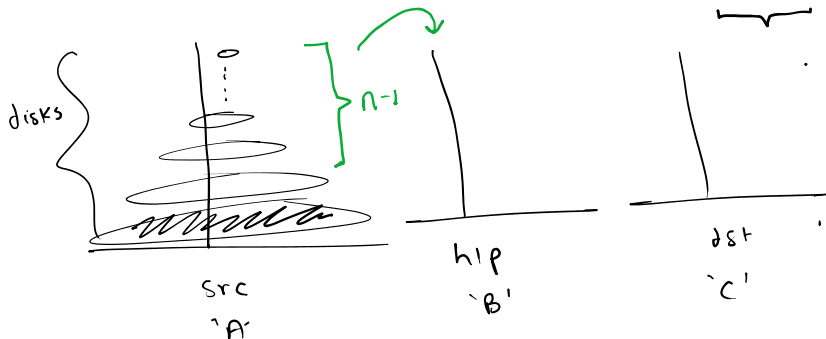
Input: $N=3$ 

o/p: {

1.	A to C	move disk from A to C
2.	A to B	move disk from A to B
3.	C to B	move disk from C to B
4.	A to C	move disk from A to C
5.	B to A	move disk from B to A
6.	B to C	move disk from B to C
7.	A to C	move disk from A to C

$n=3$ moves = 7
 $n=4$ moves = 15
 $n=5$ moves = 31

$$n \rightarrow 2^n - 1$$



→ $f(n)$ move n -disks from $src('A')$ to $dst('C')$ using $hip('B')$

$f(n-1) \rightarrow$ 1. move $n-1$ disks from $src('A')$ to $hip('B')$ using $dst('C')$

$\Rightarrow$ 2. move largest disk from src to dst
 ('A') ('B')

$f(n-1) \rightarrow$ 3. move $n-1$ disks from $hip('B')$ to $dst('C')$ using $src('A')$